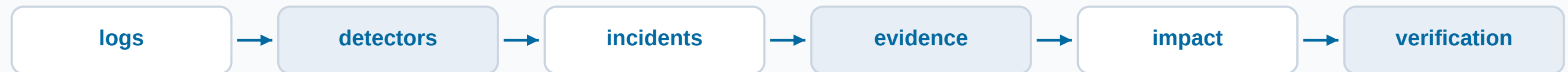


DefenceLens

Security monitoring that small organizations can actually run



HackYeah 2026 · OPEN TASK: DEFENCE

Team: Borys · Hlieb · Eduard · Aknur

github.com/NINEZERN/monitoring

The problem

The people least able to afford an incident are the least protected

- **NGOs & volunteer centers run critical infrastructure** — request portals, accounts — with zero security staff.
- **Nobody notices an attack until the service is down.** Then nobody knows what to do next.
- **Enterprise tools assume experts and money.** Per-GB pricing, multi-day setup, alert floods, opaque scores.

Small organization

1 operator, no budget,
no security training

X unreachable X

cost · complexity · expertise gap

Enterprise SIEM

analysts required
\$\$\$ per GB · weeks of tuning

Our approach

We studied 20 real weaknesses of existing tools — and built the system that closes them

Existing tools (SIEM / ELK / Wazuh / Splunk)

- ✗ Alerts that require an analyst to decode
- ✗ Alert floods, no prioritization
- ✗ Verdicts without evidence or impact
- ✗ Overconfident: “you are compromised”
- ✗ Closed scores, hidden math
- ✗ Silent data loss, lost pipelines
- ✗ Scanning sold as a separate product

DefenceLens

- ✓ Plain-language facts, limitations, next steps
- ✓ 15-min episodes with P1/P2/P3 severity
- ✓ 200 evidence events + business impact per incident
- ✓ Hypotheses, honesty, “insufficient data” verdicts
- ✓ Open, documented attention-index formula
- ✓ Idempotent ingest, disk buffers, visible counters
- ✓ Built-in Trivy scanning — image never executed

20 weaknesses of existing tools → closed (1 / 2)

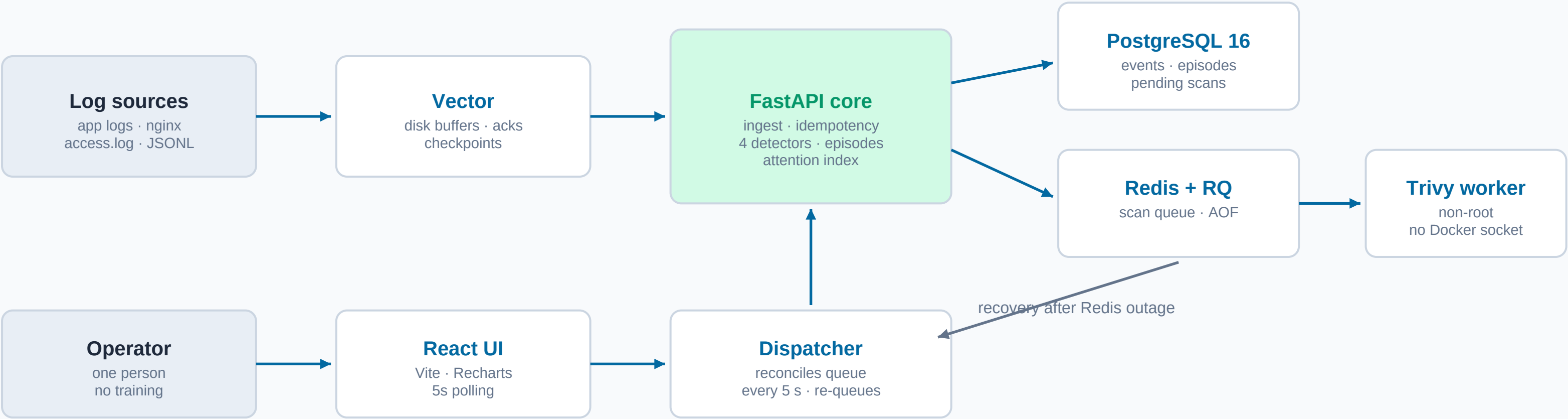
Weakness in existing solutions		DefenceLens answer	
1	Needs dedicated analysts to read alerts	→	Incidents as plain-language facts, limitations, next steps
2	Per-GB licensing costs	→	Fully open-source; runs on one small host
3	Multi-day install & tuning	→	docker compose up — minutes to a working system
4	Alert floods, no priorities	→	15-min episodes with P1/P2/P3 severity
5	Alerts without evidence	→	Up to 200 linked evidence events + timeline per incident
6	No link between alert and business impact	→	Impact view: affected service & what it blocks
7	Overconfident verdicts	→	Explicit limitations: signatures are hypotheses, not proof
8	Alert closed, problem unverified	→	Verify-outcome re-analysis; honest “insufficient data”
9	Opaque risk scores	→	Open attention-index formula (P1=25/P2=12/P3=5 × importance)
10	Silent loss on malformed logs	→	Accepted / duplicate / rejected counters; errors returned

20 weaknesses of existing tools → closed (2 / 2)

Weakness in existing solutions		DefenceLens answer	
11	Retries duplicate events	→	Idempotent ingest: unique (source_id, SHA256(event_id))
12	Pipeline loses data on restarts	→	Vector disk buffers + acks; durable scans in PostgreSQL
13	Vuln scanning is a separate product	→	Built-in Trivy: CVEs, components, potential secrets
14	Scanners trust untrusted images	→	Image never run; non-root worker, no Docker socket
15	No link from CVE to deployed fix	→	Deployment confirmation with grounds (CI digest/record)
16	Late logs break detection windows	→	Late events re-evaluate a ±10-minute window
17	Static 5xx thresholds	→	Volume + share ≥30% + growth ≥20 pp vs baseline
18	Dashboards hide the math	→	Every chart links to its calculation
19	Demo data mixed with real data	→	Synthetic loaded only explicitly, tagged; DB starts empty
20	Undefined degraded mode	→	Waiting/error/retry states; ingest continues while scans wait

System architecture

Boring, proven, fully open-source — one compose file



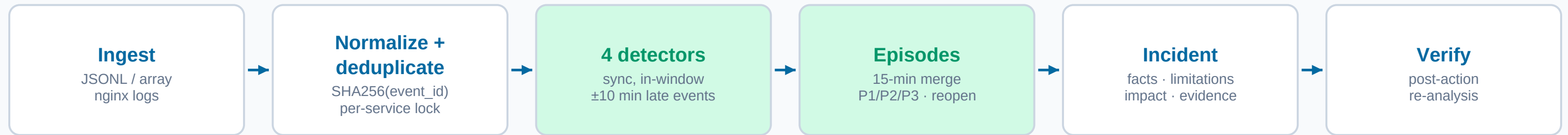
Durability: ack only after PostgreSQL commit · Vector disk buffers · Redis AOF · persistent volumes.

Security: user images are never executed · manifest validated before unpack · secret values never stored.

Tested: isolated API/rule tests · integration suite on the real stack · Playwright browser smoke test.

Detection pipeline

Deterministic and explainable — no LLM in the detection path



- **Brute force:** 5 login failures from one IP/user in 5 minutes.
- **Suspicious success:** login succeeds after ≥ 5 failures by the same pair in 5 minutes.
- **HTTP signatures:** traversal, .env/.git, SQL union select, script, wp-admin.
- **5xx surge:** ≥ 10 events / ≥ 5 errors / share $\geq 30\%$ / growth ≥ 20 pp vs previous window; missing baseline is stated.

Honest by design

The system tells the operator what it cannot know

- **Signatures are hypotheses, not proof of breach.**
- **Absence of logs does not mean absence of threats.**
- **A CVE does not prove exploitation;** temporal proximity does not prove causality.
- **“Insufficient data” is a first-class verdict,** not a hidden failure.
- **Deployment confirmation is an operator statement with grounds,** never silent automation.
- **Synthetic demo data is explicit and tagged;** the database starts empty.

Attention index — open formula, not a magic score

$P1=25 \cdot P2=12 \cdot P3=5$, $\times$ service importance/5, + up to 15 for the confirmed image; capped at 100.

Try it in 5 minutes

```
cp .env.example .env
docker compose up -d --build
```

Web UI: <http://localhost:18080> · API docs: <http://localhost:18000/docs>

Load demo → explicit synthetic data · upload your own logs or a docker save archive

No Node, Python, or Trivy needed on the host.

Built during HackYeah

FastAPI · PostgreSQL 16 · Redis/RQ
Vector · Trivy · React/Vite/Tailwind

API + rule tests · integration suite
Playwright browser smoke test

DefenceLens — defence that fits in one compose file.

Borys · Hlieb · Eduard · Aknur — HackYeah 2026 — github.com/NINEZERN/monitoring